List Data Structure

Ordered (maintains insertion order) Mutable (can be changed) Allows duplicate values Can store mixed data types

In [18]:
```
l = [] #empty list
l
```

Out[18]:
```
[]
```

In [19]:
```
type(l)
```

Out[19]:
```
list
```

In [20]:
```
l.append(10)
l
```

Out[20]:
```
[10]
```

In [21]:
```
l
```

Out[21]:
```
[10]
```

In [22]:
```
l.append(10)
l.append(20)
l.append(30)
l.append(40)
```

```
l.append(50)
l
```

Out[22]: [10, 10, 20, 30, 40, 50]

In [23]:
```
len(l)
```

Out[23]: 6

In [24]:
```
l.append(10)
l
```

Out[24]: [10, 10, 20, 30, 40, 50, 10]

In [25]:
```
l1 = [10,2.3, 1+2j, True, 'nit',[1,2,3]]
l1
```

Out[25]: [10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]

In [26]:
```
len(l1)
```

Out[26]: 6

In [27]:
```
print(l)
print(l1)
```

```
[10, 10, 20, 30, 40, 50, 10]
[10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [28]:
```
print(id(l))
```

```
print(id(l1))
```

```
2262230871744
2262230792768
```

In [29]:
```
l #l is called variable (10,20,30 are value or element)
```

Out[29]:
```
[10, 10, 20, 30, 40, 50, 10]
```

In [30]:
```
l2 = l.copy()
l2
```

Out[30]:
```
[10, 10, 20, 30, 40, 50, 10]
```

In [31]:
```
l2
```

Out[31]:
```
[10, 10, 20, 30, 40, 50, 10]
```

In [32]:
```
len(l2)
```

Out[32]:
```
7
```

In [33]:
```
l2.clear()
```

In [34]:
```
len(l2)
```

Out[34]:
```
0
```

In [35]:
```
l2
```

Out[35]: []

In [36]:
```
del l2
l2
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[36], line 2
      1 del l2
----> 2 l2

NameError: name 'l2' is not defined
```

In [ ]:
```
l
```

In [ ]:
```
l.count(20)
```

In [37]:
```
l.count(10)
```

Out[37]: 3

In [38]:
```
l.count(100)
```

Out[38]: 0

In [39]:
```
print(l)
print(l1)
```

```
[10, 10, 20, 30, 40, 50, 10]
[10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [40]:
```
a= [1,2,3]
b= [4,5,6]

a.extend(b)
a
```

Out[40]:
```
[1, 2, 3, 4, 5, 6]
```

In [41]:
```
l2 = []
l2
```

Out[41]:
```
[]
```

In [42]:
```
l2.extend(l)
l2
```

Out[42]:
```
[10, 10, 20, 30, 40, 50, 10]
```

In [43]:
```
l2.extend(l1)
l2
```

Out[43]:
```
[10, 10, 20, 30, 40, 50, 10, 10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [44]:
```
print(l)
print(l1)
print(l2)
```

```
[10, 10, 20, 30, 40, 50, 10]
[10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
[10, 10, 20, 30, 40, 50, 10, 10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [45]:
```
l[0]
```

Out[45]:
```
10
```

In [46]:
```
l[0]= 100 #list is muteable
l
```

Out[46]:
```
[100, 10, 20, 30, 40, 50, 10]
```

In [47]:
```
l[-2]= 500 #mutable
l
```

Out[47]:
```
[100, 10, 20, 30, 40, 500, 10]
```

In [48]:
```
l.index(100)
```

Out[48]:
```
0
```

In [49]:
```
l2
```

Out[49]:
```
[10, 10, 20, 30, 40, 50, 10, 10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [50]:
```
l2.index(10)
```

Out[50]:
```
0
```

In [51]:
```
l.insert(2, 25)
l
```

Out[51]: [100, 10, 25, 20, 30, 40, 500, 10]

In [52]:
```
l2
```

Out[52]: [10, 10, 20, 30, 40, 50, 10, 10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]

In [53]:
```
l2.pop()
```

Out[53]: [1, 2, 3]

In [54]:
```
l2
```

Out[54]: [10, 10, 20, 30, 40, 50, 10, 10, 2.3, (1+2j), True, 'nit']

In [55]:
```
l2.pop()
```

Out[55]: 'nit'

In [56]:
```
l2.pop()
```

Out[56]: True

In [57]:
```
l2
```

Out[57]: [10, 10, 20, 30, 40, 50, 10, 10, 2.3, (1+2j)]

In [58]:
```
l2.pop(2)
l2
```

Out[58]: [10, 10, 30, 40, 50, 10, 10, 2.3, (1+2j)]

In [59]:
```
l2.remove(40)
l2
```

Out[59]: [10, 10, 30, 50, 10, 10, 2.3, (1+2j)]

In [60]:
```
l2.pop(4)
```

Out[60]: 10

In [61]:
```
l.reverse()
l
```

Out[61]: [10, 500, 40, 30, 20, 25, 10, 100]

In [62]:
```
l.sort()
l
```

Out[62]: [10, 10, 20, 25, 30, 40, 100, 500]

In [63]:
```
l
```

Out[63]: [10, 10, 20, 25, 30, 40, 100, 500]

In [64]:
```
l.sort(reverse=True)
l
```

Out[64]: [500, 100, 40, 30, 25, 20, 10, 10]

In [65]:
```
l3 = [10,20,[2,3],3]
```

In [66]:
```
l3[2] = [2]
l3
```

Out[66]:
```
[10, 20, [2], 3]
```

In [67]:
```
l
```

Out[67]:
```
[500, 100, 40, 30, 25, 20, 10, 10]
```

In [68]:
```
30 in l
```

Out[68]:
```
True
```

In [69]:
```
500 in l
```

Out[69]:
```
True
```

In [70]:
```
5000 in l  #membership
```

Out[70]:
```
False
```

In [71]:
```
for i in l:
    print(i)
```

```
500
100
40
30
25
20
10
10
```

In [72]:
```
l.remove(30) #Remove by value
l.pop()      #Remove last element
l.pop(1)     #Remove by index
```

Out[72]:
```
100
```

In [73]:
```
l3 = [1, 2, 3, 4, 5]
print(l3[1:4])  # [2, 3, 4]
```

```
[2, 3, 4]
```

In [74]:
```
squares = [x*x for x in range(5)]
print(squares)
```

```
[0, 1, 4, 9, 16]
```

In [75]:
```
matrix = [[1, 2], [3, 4]]
print(matrix[0][1]) # 2
```

```
2
```

In [76]:
```
students = ["Alice", "Bob" , "Charlie"]
students.append("David")
```

```
students
```

Out[76]: ['Alice', 'Bob', 'Charlie', 'David']

In [77]:
```
def add_item(item, lst=[]):
    lst.append(item)
    return lst

print(add_item(1))
print(add_item(2))
print(add_item(3))
```

```
[1]
[1, 2]
[1, 2, 3]
```

In [78]:
```
lst = [[0]*3]*3
print(lst)
lst[0] [0] = 1
print(lst)
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

In [79]:
```
import copy

a = [[1, 2], [3, 4]]
b = copy.copy(a)
print(b)
b[0][0] = 99
print(a)
```

```
[[1, 2], [3, 4]]
[[99, 2], [3, 4]]
```

- Shallow Copy vs Deep Copy

In [80]:
```
nums = [1, 2, 3, 4, 5, 6]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)
print(nums)
```

```
[1, 3, 5]
```

- Removing While Iterating

In [81]:
```
nums = [1, 2, 3, 4, 5, 6]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)
print(nums)
```

```
[1, 3, 5]
```

- List vs Reference

In [82]:
```
a = [1, 2, 3]
b = a
b.append(4)
```

```
print(a)
```

```
[1, 2, 3, 4]
```

- Slicing Copy Trick

In [83]:
```
a = [1, 2, 3]
b = a[:]
b.append(4)

print(a)
print(b)
```

```
[1, 2, 3]
[1, 2, 3, 4]
```

- Nasted List Modification

In [84]:
```
a = [[1, 2], [3,4]]
b = a[:]

b[0][0] = 100
print(a)
```

```
[[100, 2], [3, 4]]
```

In [85]:
```
a = [[1, 2], [3, 4]]
b = a[:]
```

```
b[0][0] = 100
print(a)
```

```
[[100, 2], [3, 4]]
```

- Sorting Trick

In [86]:
```
a = [3, 1, 2]
b = a.sort()

print(a)
print(b)
```

```
[1, 2, 3]
None
```

- Extend vs Append

In [87]:
```
a = [1, 2]
a.append([3, 4])
print(a)

b = [1, 2]
b.extend([3, 4])
print(b)
```

```
[1, 2, [3, 4]]
[1, 2, 3, 4]
```

- List Comprehension Scope

In [88]:
```
x = 10
lst = [x for x in range(5)]
print(x)
```

```
10
```

- Identity vs Equality- == compares values is campares memory location

In [89]:
```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)
print(a is b)
```

```
True
False
```

- PDF

In [90]:
```
txt = " abc def ghi "

print(txt.lstrip() )
print(txt.rstrip() )
print(txt.strip() )
```

```
abc def ghi
 abc def ghi
abc def ghi
```

- Using Escape Character

In [91]:
```
#Using double quotes in the string is not allowed.
mystr = "My favourite TV Series is "Game of Thrones""
```

```
  Cell In[91], line 2
    mystr = "My favourite TV Series is "Game of Thrones""
                                        ^
SyntaxError: invalid syntax
```

In [92]:
```
#Using escape charater to allow illegal character
mystr = "My favourite series is \"Game of Thrones\""
print(mystr)
```

```
My favourite series is "Game of Thrones"
```

- List Creation

In [93]:
```
listl = [] #Empty List
```

In [94]:
```
print(type(listl))
```

```
<class 'list'>
```

In [95]:
```
list2 = [10,30,60] # List of integers numbers
```

In [96]:
```
list3 = [10.77,30.66,60.89] #List of float numbers
```

In [97]:
```
list4 = ['one','two',"three"] #List of strings
```

In [98]:
```
list5 = ['Asif', 25 , [50, 100],[150, 90]] #Nested List
```

In [99]:
```
list6 = [100, 'Asif', 17.765] #List of mixed data types
```

In [100…
```
liste7 = ['Asif', 25 , [50, 100],[150, 90] , {'John' , 'David'}]
```

In [101…
```
len(list6) #Length of List
```

Out[101…
```
3
```

In [102…
```
list2[0] #Retreive first element of the list
```

Out[102…
```
10
```

In [103…
```
list4[0] #Retreive first element of the list
```

Out[103…
```
'one'
```

In [104…
```
list4[0][0] #Nested indexing - Access the first character of the first list element
```

Out[104…
```
'o'
```

In [105…
```
list4[-1] #Lat item of the list
```

Out[105…
```
'three'
```

In [106…
```
list5[-1] #last item of list
```
Out[106…
```
[150, 90]
```
In [107…
```
mylist = ['one' , 'two', 'three', 'four', 'five' , 'six','seven','eight']
```
In [108…
```
mylist[0:3] #Return all items from 0th to 3rd index location excluding the item
```
Out[108…
```
['one', 'two', 'three']
```
In [109…
```
mylist[2:5] # List all items from 2nd to 5th index location excluding the items a
```
Out[109…
```
['three', 'four', 'five']
```
In [110…
```
mylist[:3] # Return first three items
```
Out[110…
```
['one', 'two', 'three']
```
In [111…
```
mylist[:2] # Return first two items
```
Out[111…
```
['one', 'two']
```
In [112…
```
mylist[-3:] # Return first two items
```
Out[112…
```
['six', 'seven', 'eight']
```
In [113…
```
mylist[-2:] # Return last two items
```

Out[113… ['seven', 'eight']

In [114…
```
mylist[-1] # Return last item of the list
```

Out[114… 'eight'

In [115…
```
mylist[:] # Return whole list
```

Out[115… ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']

Add , Remove & Change Items

In [116…
```
mylist
```

Out[116… ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']

In [117…
```
mylist.append('nine') # Add on item to the end of the list
mylist
```

Out[117… ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [118…
```
mylist.insert(9,'ten') # Add item an index location 9
mylist
```

Out[118… ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']

In [119…

```
mylist.insert(1, 'ONE') # Add item at index location 1
mylist
```

Out[119…

```
['one',
 'ONE',
 'two',
 'three',
 'four',
 'five',
 'six',
 'seven',
 'eight',
 'nine',
 'ten']
```

In [120…

```
mylist.remove('ONE') # Remove item "ONE"
mylist
```

Out[120…

```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
```

In [121…

```
mylist.pop() # Remove last item of the list
mylist
```

Out[121…

```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [122…

```
mylist.pop(8) # remove item at index location 8
mylist
```

Out[122…

```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

In [123…

```
del mylist[7] # Remove item at index location 7
mylist
```

Out[123…

```
['one', 'two', 'three', 'four', 'five', 'six', 'seven']
```

In [124…

```
mylist[0] = 1
mylist[1] = 2
mylist[2] = 3
mylist
```

Out[124…

```
[1, 2, 3, 'four', 'five', 'six', 'seven']
```

In [125…

```
mylist.clear() # Empty List / Delete all items in the list
mylist
```

Out[125…

```
[]
```

In [126…

```
del mylist # Delete the whole list
mylist
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[126], line 2
      1 del mylist # Delete the whole list
----> 2 mylist

NameError: name 'mylist' is not defined
```

- Copy List

In [ ]:
```
mylist = ['one','two','three','four','five','six','seven','eight','nine']
```

In [ ]:
```
mylist = mylist # create a new reference "mylist1"
```

In [127…
```
id(mylist) , id(mylist) # The address of both mylist & mylist will be the same
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[127], line 1
----> 1 id(mylist) , id(mylist) # The address of both mylist & mylist will be the same

NameError: name 'mylist' is not defined
```

In [ ]:
```
mylist2 = mylist.copy() # Create a copy of the list
```

In [ ]:
```
id(mylist2) # The adress of mylist2 will be different from mylist beacuse mylist
```

In [128…
```
mylist[0] = 1
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[128], line 1
----> 1 mylist[0] = 1

NameError: name 'mylist' is not defined
```

In [ ]:
```
mylist
```

In [ ]:
```
mylist1 # mylist1 will be also impacted as it is pointing to the same list
```

In [129…

```python
mylist2 # Copy of list won't be impacted due to changes made on the orginal
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[129], line 1
----> 1 mylist2 # Copy of list won't be impacted due to changes made on the orginal

NameError: name 'mylist2' is not defined
```

In [ ]:

```python
list1 = ['one','two','three','four']
list2 = ['five', 'six','seven','eight']
```

In [ ]:

```python
list1 = list1 + list2 # Join two list by '+' operator
list1
```

In [130…

```python
list1.extend(list2) #Append list2 with list1
list1
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[130], line 1
----> 1 list1.extend(list2) #Append list2 with list1
      2 list1

NameError: name 'list1' is not defined
```

In [ ]:

```python
list1
```

In [ ]:

```python
'one' in list1 # Check if 'one' exist in the list
```

In [131…

```
'ten' in list1 # Check if 'ten' exist in the list
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[131], line 1
----> 1 'ten' in list1 # Check if 'ten' exist in the list

NameError: name 'list1' is not defined
```

In [ ]:

```
if 'three' in list1: # Check if 'three' exist in the list
    print('Three is present in the list')
else:
    print('Three is not present in the list')
```

In [ ]:

```
if 'evelen' in list1: # Check if 'elevn' exist in the list
    print('eveln is present in the list')
else:
    print('eleven is not present in the list')
```

Revese & Sort List

In [132…

```
list1
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[132], line 1
----> 1 list1

NameError: name 'list1' is not defined
```

In [ ]:
```
list1.reverse() # Reverse the list
list1
```

In [ ]:
```
list1 = list1[::-1] # Reverse the list
list1
```

In [133…
```
mylist3 = [9,5,2,99,12,88,34]
mylist3.sort() # Sort list in ascending order
mylist3
```

Out[133…
```
[2, 5, 9, 12, 34, 88, 99]
```

In [134…
```
mylist3 = [9,5,2,99,12,88,34]
mylist3.sort(reverse=True) # Sort list in decending order
mylist3
```

Out[134…
```
[99, 88, 34, 12, 9, 5, 2]
```

In [135…
```
mylist4 = [88,65,33,21,11,98]
sorted(mylist4) #Returns a new sorted list and doesn't change original
```

Out[135…
```
[11, 21, 33, 65, 88, 98]
```

In [136…
```
mylist4
```

Out[136…
```
[88, 65, 33, 21, 11, 98]
```

In [137…
```
l3 = [3, 100, 2, 50]
l3
```

Out[137…
```
[3, 100, 2, 50]
```

In [138…
```
l3.sort()
l3
```

Out[138…
```
[2, 3, 50, 100]
```

In [139…
```
l3.sort(reverse = False #parameter tunning
l3
```

```
  Cell In[139], line 1
    l3.sort(reverse = False #parameter tunning
            ^
SyntaxError: '(' was never closed
```

In [ ]:
```
l3.sort(reverse = True) #hyperparameter tunning
l3
```

In [140…
```
l4 = [2, 'a', 2.4, True]
l4
```

Out[140…
```
[2, 'a', 2.4, True]
```

In [141…
```
l4.sort() #we cant sort number with sort
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[141], line 1
----> 1 l4.sort() #we cant sort number with sort

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [ ]:

List Indexing Slicing

In [142…
```
l1[:]
```

Out[142…
```
[10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [143…
```
l1[::-2]
```

Out[143…
```
[[1, 2, 3], True, 2.3]
```

In [144…
```
l1
```

Out[144…
```
[10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

In [145…
```
l1[0:10:3]
```

Out[145… [10, True]

In [146…
```
l1[2] = 45
l1
```

Out[146… [10, 2.3, 45, True, 'nit', [1, 2, 3]]

List datastructure completed

Tuple or Tuple

In [147…
```
t = ()
t
```

Out[147… ()

In [148…
```
type(t)
```

Out[148… tuple

In [149…
```
t1 = (10,20,30)
t1
```

Out[149… (10, 20, 30)

In [150…
```
t1.count(10)
```
Out[150…
```
1
```
In [151…
```
t1.count(100)
```
Out[151…
```
0
```
In [152…
```
t1.index(10)
```
Out[152…
```
0
```
In [153…
```
t1
```
Out[153…
```
(10, 20, 30)
```
In [154…
```
t1[0] #mutable
```
Out[154…
```
10
```
In [155…
```
t1[0] = 100 #immutable
```
```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[155], line 1
----> 1 t1[0] = 100

TypeError: 'tuple' object does not support item assignment
```

In [156…
```
l
```
Out[156…
```
[500, 40, 25, 20, 10]
```
In [157…
```
t1
```
Out[157…
```
(10, 20, 30)
```
In [158…
```
t1 * 3 #repeat 3 times
```
Out[158…
```
(10, 20, 30, 10, 20, 30, 10, 20, 30)
```
In [161…
```
l7 = [10,20,40]
l7 * 2
```
Out[161…
```
[10, 20, 40, 10, 20, 40]
```
In [ ]:
```

```

list & tuple is completed

SET

In [162…
```
s = {} #empty dict
s
```

Out[162… {}

In [163…
```
type(s)
```

Out[163… dict

In [164…
```
s1 = set() #empty set
s1
```

Out[164… set()

In [165…
```
s1 = {20,30,90,10} #set is in ordered accending order
s1
```

Out[165… {10, 20, 30, 90}

In [166…
```
type(s1)
```

Out[166… set

In [167…
```
s1
```

Out[167… {10, 20, 30, 90}

In [168…
```
s1.add(100)
s1
```

Out[168… {10, 20, 30, 90, 100}

In [170…

```
s1.add(15) #in ordered
s1
```

Out[170…

```
{10, 15, 20, 30, 90, 100}
```

In [171…

```
s1.add(5)
s1
```

Out[171…

```
{5, 10, 15, 20, 30, 90, 100}
```

In [172…

```
s2 = {5, 5.5,'nit',1+2j,True}
s2
```

Out[172…

```
{(1+2j), 5, 5.5, True, 'nit'}
```

In [174…

```
s2.add(100)
s2
```

Out[174…

```
{(1+2j), 100, 5, 5.5, True, 'nit'}
```

In [175…

```
print(s1)
print(s2)
```

```
{100, 5, 10, 15, 20, 90, 30}
{'nit', True, 100, 5.5, 5, (1+2j)}
```

In [176…

```
s3 = s2.copy()
s3
```

Out[176…

```
{(1+2j), 100, 5, 5.5, True, 'nit'}
```

In [178…

```
s3[0]
s3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[178], line 1
----> 1 s3[0]
      2 s3

TypeError: 'set' object is not subscriptable
```

In [179…

```
s3[:]
s3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[179], line 1
----> 1 s3[:]
      2 s3

TypeError: 'set' object is not subscriptable
```

In [182…

```
s3.pop()
```

Out[182…

```
100
```

In [183…

```
s3
```

Out[183…

```
{(1+2j), 5, 5.5}
```

In [184…
```
s3.pop(3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[184], line 1
----> 1 s3.pop(3)

TypeError: set.pop() takes no arguments (1 given)
```

In [185…
```
s2
```

Out[185…
```
{(1+2j), 100, 5, 5.5, True, 'nit'}
```

In [186…
```
s1
```

Out[186…
```
{5, 10, 15, 20, 30, 90, 100}
```

In [187…
```
s2
```

Out[187…
```
{(1+2j), 100, 5, 5.5, True, 'nit'}
```

In [189…
```
s2.discard(1+2j)
s2
```

Out[189…
```
{100, 5, 5.5, True, 'nit'}
```

In [190…
```
s2.discard(1000)  # no error
s2
```

Out[190…

```
{100, 5, 5.5, True, 'nit'}
```

In [191…

```
s2.remove(1000) # give error
s2
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[191], line 1
----> 1 s2.remove(1000)
      2 s2

KeyError: 1000
```

In [192…

```
s2
```

Out[192…

```
{100, 5, 5.5, True, 'nit'}
```

In [193…

```
s2.add(100)
s2
```

Out[193…

```
{100, 5, 5.5, True, 'nit'}
```

In [194…

```
s2
```

Out[194…

```
{100, 5, 5.5, True, 'nit'}
```

SET OPERATION -->

In [196…

```
s7 = {1,2,3,4,5}
s8 = {4,5,6,7,8}
s9 = {8,9,10}
```

In [197…

```
s7.union(s8)
```

Out[197…

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

In [198…

```
s7.union(s8,s9)
```

Out[198…

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

In [199…

```
s8 | s9
```

Out[199…

```
{4, 5, 6, 7, 8, 9, 10}
```

In [200…

```
s8 | s9 | s7
```

Out[200…

```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

In [201…

```
print(s7)
print(s8)
print(s9)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7, 8}
{8, 9, 10}
```

In [ ]:

In [ ]: